

Anime Vector Space Model – Semantic Search

Đồ án cuối kỳ NLP: xây dựng **Vector Space Model** cho tìm kiếm ngữ nghĩa trên dataset Anime (Kaggle), lưu trữ embedding trong **Milvus**, và **so sánh 2 embedding model**:

- `intfloat/e5-small-v2` — 384 chiều, dùng prefix `query: / passage:`
- `sentence-transformers/all-MiniLM-L6-v2` — 384 chiều, không cần prefix

Toàn bộ phân tích nằm ở **Mục 3** trong `vector-space-model.ipynb` , kèm một **FastAPI** (`apps/`) để import dữ liệu và truy vấn.

1. Yêu cầu hệ thống

Thành phần	Phiên bản
Docker Desktop	đang chạy (cho Milvus)
Python	3.12
RAM trống	≥ 4 GB (Milvus + model)

2. Cấu trúc dự án

```
final-project/
├── docker-compose.yml      # Milvus (etcd + minio + standalone) + Attu UI
├── requirements.txt
├── .env                    # HF_TOKEN, cấu hình Milvus (KHÔNG commit)
├── vector-space-model.ipynb # Mục 3: phân tích, truy vấn, đánh giá, so sánh
├── kangle/dataset/         # anime.csv, rating.csv (dữ liệu Kaggle)
├── models/                 # 2 model tải về (tự sinh, đã .gitignore)
├── apps/                   # FastAPI + service dùng chung
│   ├── main.py             # khởi tạo app, endpoint /, /models
│   ├── config.py           # MODEL_REGISTRY, Settings (.env)
│   ├── cli.py              # pull-models / import-data / stats
│   ├── services/           # data_loader, embedding, milvus_store, search, evaluat
│   ├── controller/         # nghiệp vụ import & search
│   └── router/             # /import, /search, /search/similar, /search/compare
```

3. Cài đặt

3.1. Tạo môi trường Python & cài thư viện

```
python3.12 -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

3.2. Cấu hình .env

Tạo file `.env` ở thư mục gốc (xem mẫu `.env.example`):

```
HF_TOKEN=hf_XXXXXXXXXXXXXXXXXXXXXXX
```

HF_TOKEN dùng để làm gì? Token được truyền vào

`SentenceTransformer(..., token=HF_TOKEN)` khi tải model

(`apps/services/embedding.py`). Hai model trong đồ án là **public** nên token không bắt buộc, nhưng nên có để

tránh bị giới hạn tốc độ tải (rate-limit) của HuggingFace và để dùng được model private nếu

cần.

Các tham số Milvus (MILVUS_HOST , MILVUS_PORT) đã có giá trị mặc định localhost:19530 .

3.3. Khởi động Milvus (+ Attu UI)

```
docker compose up -d
```

Dịch vụ được tạo: milvus-etcd , milvus-minio , milvus , milvus-attu .

4. Chạy 1 lần: tải model & import dữ liệu

```
python -m apps.cli pull-models      # tải 2 model về thư mục models/
python -m apps.cli import-data      # encode 12.294 anime -> nạp vào Milvus (cả 2 model)
python -m apps.cli import-data -f   # ép nạp lại (xoá dữ liệu cũ rồi nạp mới)
python -m apps.cli stats             # kiểm tra số vector trong từng collection
```

Idempotent — không lo trùng dữ liệu: import-data mặc định **bỏ qua** model nào đã có dữ liệu trong Milvus (và không tốn công load CSV/encode). Chỉ khi collection rỗng nó mới import.

Muốn nạp lại từ đầu thì dùng cờ `-f/--force` .

Notebook cũng **không tự re-import**: cell ở Mục 3.0 chỉ import khi collection rỗng (đặt `FORCE_IMPORT = True` nếu muốn ép trong notebook).

Sau khi import, mỗi model có 1 collection riêng:

Model	Collection	Số vector
e5-small-v2	anime_e5_small_v2	12.294
all-MiniLM-L6-v2	anime_minilm_l6_v2	12.294

5. Xem dữ liệu đã import & các điểm vector

Attu — Web UI của Milvus (khuyến nghị)

Mở trình duyệt: <http://localhost:8001>

- Ở màn hình kết nối, nhập địa chỉ Milvus: `milvus:19530` (hoặc `localhost:19530` nếu chạy ngoài Docker), bỏ trống user/password → **Connect**.
- Vào **Collections** → chọn `anime_e5_small_v2` hoặc `anime_minilm_l6_v2` để xem schema, số bản ghi.
- Tab **Data** xem từng bản ghi (`anime_id`, `name`, `genre`, `vector`...).
- Tab **Vector Search** dán 1 vector để thử tìm kiếm trực tiếp trong UI.

MinIO Console — nơi Milvus lưu file vector

Mở: <http://localhost:9001> — đăng nhập `minioadmin` / `minioadmin` để xem các segment/log mà Milvus ghi xuống object storage.

6. Chạy phân tích (Mục 3 — Notebook)

```
jupyter lab          # hoặc mở vector-space-model.ipynb trong VS Code / Cursor
```

Chọn kernel `.venv` rồi chạy **Mục 3**. Nội dung:

- **3.0** Thiết lập & nạp dữ liệu
- **3.1** Tiền xử lý (document, phân bố độ dài)
- **3.2** Sinh embedding & kho vector (tốc độ encode)
- **3.3** Truy vấn free-text → xếp hạng; tìm anime tương tự; ma trận cosine
- **3.4** Đánh giá (Precision@K, Recall@K, MRR, nDCG@K) & trực quan hoá
- **3.5** Nhận xét & kết luận so sánh

7. Chạy FastAPI (import & research)

```
uvicorn apps.main:app --reload
```

Swagger UI: <http://localhost:8000/docs>

Method	Endpoint	Mô tả
GET	/	Trạng thái + số vector mỗi collection
GET	/models	Danh sách model
POST	/import	Encode & nạp anime vào Milvus (idempotent; {"force": true} để nạp lại)
GET	/import/stats	Số vector hiện có
POST	/search	Truy vấn free-text
POST	/search/similar	Tìm anime tương tự (more-like-this)
POST	/search/compare	So sánh kết quả 2 model

Ví dụ:

```
curl -X POST http://localhost:8000/search \  
-H "Content-Type: application/json" \  
-d '{"query": "epic fantasy adventure with magic", "model": "e5-small-v2", "top_k": 5
```

8. Kết quả so sánh (300 query, Top-K=10)

Model	Precision@10	Recall@10	MRR	nDCG@10	ms/query
e5-small-v2	0.377	0.052	0.724	0.424	2.28
all-MiniLM-L6-v2	0.260	0.036	0.626	0.316	1.11

→ e5-small-v2 cho **chất lượng truy hồi tốt hơn**; all-MiniLM-L6-v2 **nhẹ hơn ~2x**.

9. Dừng dịch vụ

```
docker compose down          # giữ dữ liệu (volumes/)  
docker compose down -v      # xoá luôn dữ liệu Milvus
```